

- JS1-Promise-题解
 - 1 题目ID-4
 - 2 题目ID-14

JS1-Promise-题解

1 题目ID-4

```
setTimeout(function () {  
    console.log(1);  
}, 0);  
new Promise(function executor(resolve) {  
    console.log(2);  
    for (var i = 0; i < 10000; i++) {  
        i == 9999 && resolve();  
    }  
    console.log(3);  
}).then(function () {  
    console.log(4);  
});  
console.log(5);
```

知识点

执行顺序

输出顺序

2 3 5 4 1

解析

第一步

首先执行整个脚本，这本身是一个宏任务。

1. 遇到setTimeout，注册宏任务A(console.log(1))。
2. 执行Promise内的同步代码，输出2 3。
3. 执行Promise.then(……)，注册微任务P(console.log(4))。
4. 执行同步代码，输出5。

控制台: 2 3 5
宏任务: [A(console.log(1))]
微任务: [P(console.log(4))]

第二步

清空微任务队列。
执行微任务P，打印4。

控制台: 2 3 5 4
宏任务: [A(console.log(1))]
微任务: []

第三步

执行宏任务A，打印1。

控制台: 2 3 5 4 1
宏任务: []
微任务: []

2 题目ID-14

```
console.log('start')
async function ac () {
  console.log('ac-start');
  await new Promise((resolve) => {
    resolve('p3');
  }).then((res) => {
    console.log(res);
  });
  await new Promise((resolve) => {
    console.log('p1');
    return 'p2';
  })
}
```

```
    console.log('success');  
    return 'ac-end';  
}  
ac().then(res => console.log(res));  
console.log('end');
```

知识点

执行顺序

输出顺序

start ac-start end p3 p1

解析

第一步

首先执行整个脚本，这本身是一个宏任务。

1. 输出start
2. 遇到`ac().then(.....)`
 - 2.1 执行同步代码，输出ac-start
 - 2.2 遇到`await new Promise`
 - 2.2.1 执行Promise代码`resolve('p3')`，状态变为fulfilled。
 - 2.2.2 由于Promise已解决，回调被放入微任务队列。注册微任务`P1(console.log('p3'))`。
 - 2.2.3 `await`后的代码暂停执行。
3. 输出end

控制台: start ac-start end
微任务: [P1(console.log('p3'))]
宏任务: []

第二步

清空微任务队列。

1. 执行微任务P1，输出p3。
2. `await`等待的Promise状态变为resolved。
3. 引擎将`async`函数中第一个`await`之后的剩余代码包装成一个新的微任务P2。

控制台: start ac-start end p3
微任务: [P2]

宏任务: []

第三步

清空微任务队列。

1. 执行微任务P2。

2. 遇到await new Promise

2.1 执行Promise代码, 输出p1, 执行 return 'p2' (无效, 不改变 Promise 状态)

2.2 await 遇到 pending 状态的 Promise, 再次暂停 async 函数

没有调用 resolve, 因此第二个 await 返回的 Promise 永远处于 pending, 导致不会将后续代码加入微任务队列, 且ac().then(...)也永远不会执行。

控制台: start ac-start end p3 p1

微任务: []

宏任务: []